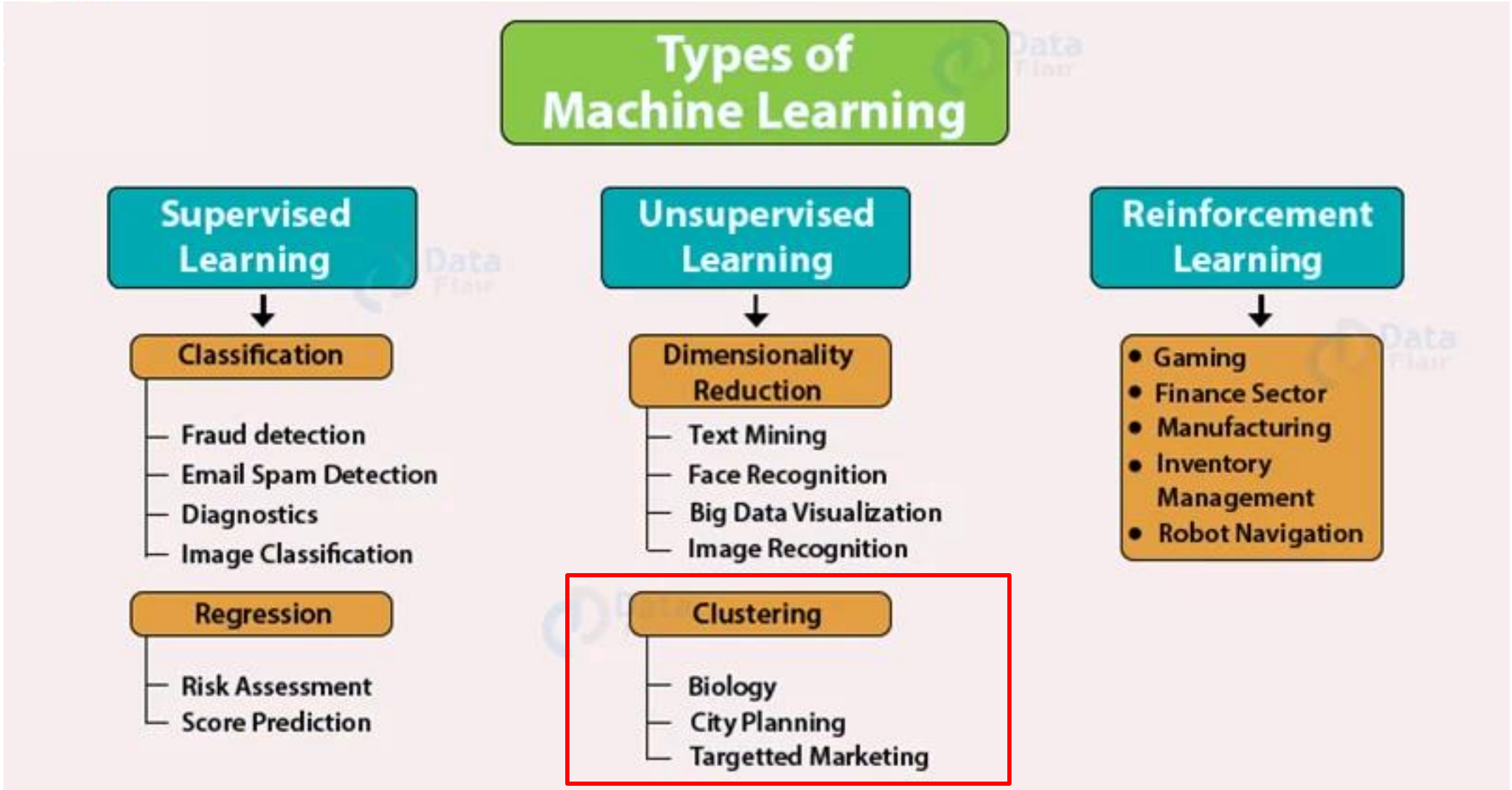
Unsupervised Learning
“Clustering”

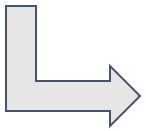

Types of Machine Learning
Supervised Learning
Classification
Fraud detection
Email Spam Detection
Diagnostics
Image Classification
Regression
Risk Assessment
Score Prediction
Unsupervised Learning
Dimensionality Reduction
Text Mining
Face Recognition
Big Data Visualization
Image Recognition
Clustering
Biology
City Planning
Targetted Marketing
Reinforcement Learning
Gaming
Finance Sector
Manufacturing
Inventory Management
Robot Navigation

When Dealing with Real-World Problems

Most of the time, data **does not come** with **predefined labels**!!!

What can we do?

We can develop machine learning models that can correctly classify **unlabeled data** by autonomously **identifying commonalities** in the features. These commonalities are then used to predict classes for new data.

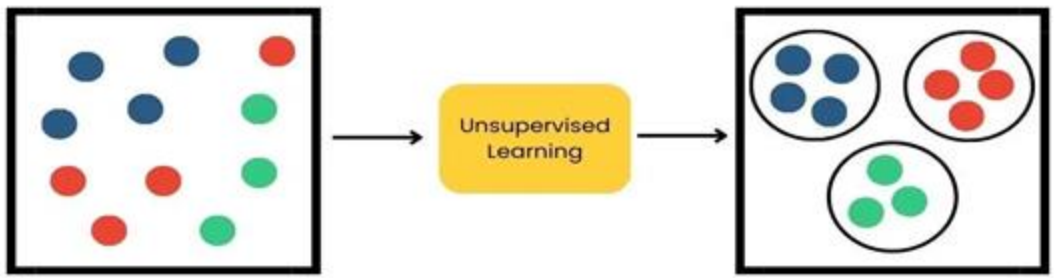


Unsupervised Learning

Definition: Unsupervised Learning

Unsupervised learning, also known as unsupervised machine learning, uses machine learning (ML) algorithms to analyze and **cluster unlabeled data sets**.

- These algorithms discover hidden patterns or data groupings without the need for human intervention. **E.g. Clustering**.



The main goal of these types of algorithms is to **study the intrinsic and hidden structure of the data** in order to *get meaningful insights*, segment the datasets in **similar groups or to simplify them**

Some Applications

- **Cluster Analysis:** Segmenting datasets by shared attributes to identify natural groupings or clusters within the data.

- **Dimensionality Reduction:** Simplifying datasets by aggregating variables with similar attributes, thereby reducing the number of dimensions while preserving the essential structure of the data.

- **Anomaly Detection:** Identifying anomalies or outliers in the data that do not conform to the patterns exhibited by the majority of the dataset.

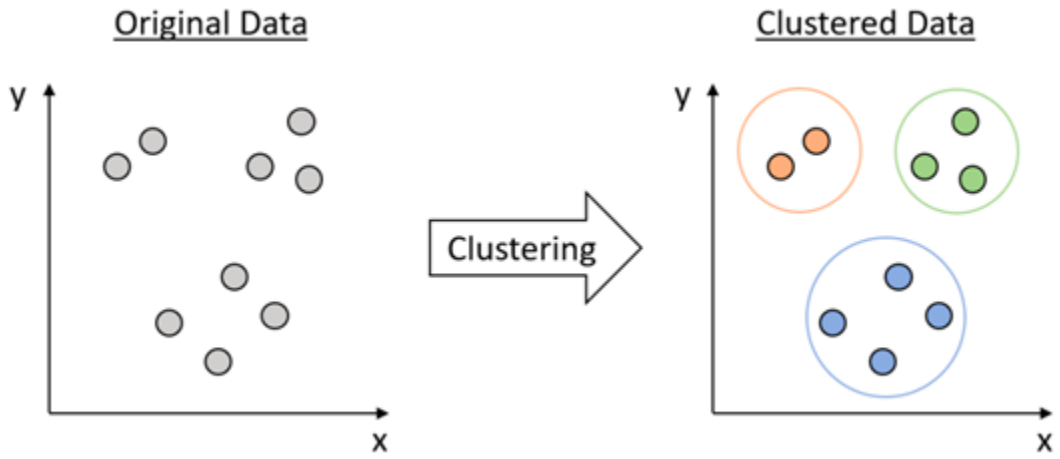
What can we Cluster in Practice?

- News articles or web pages by topic
- Protein sequences by function, or genes according to expression profile
- Users of social networks by interest
- Customers according to purchase history

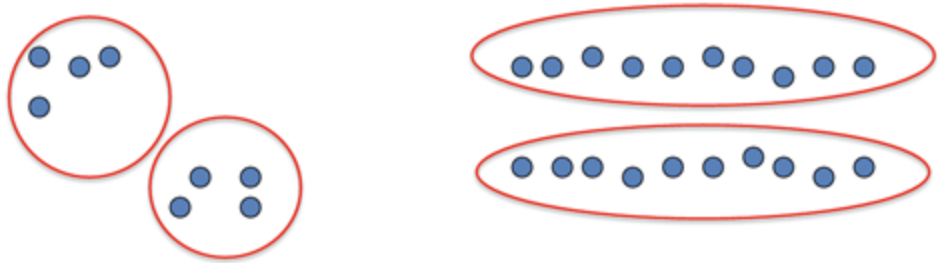
Clustering

Cluster analysis is a powerful unsupervised learning technique used to identify natural groupings or clusters within datasets.

- **Group similar data points** or objects into clusters or categories.
- By grouping data points with similar characteristics, cluster analysis helps **reveal hidden patterns, trends, and relationships**.



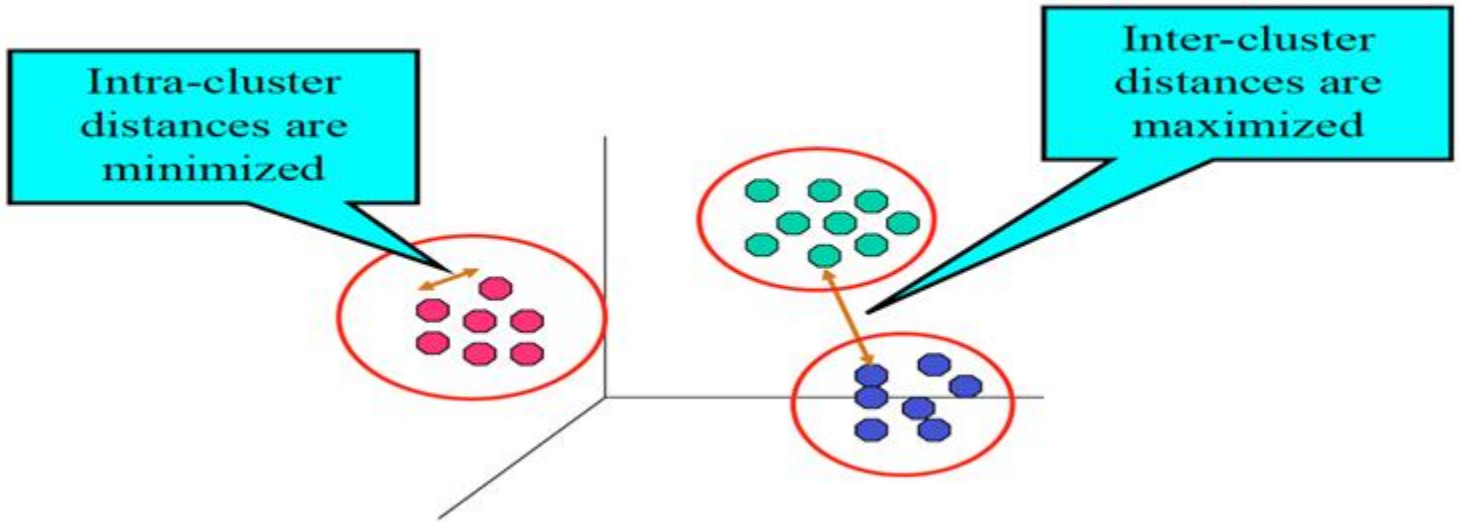
- Basic idea: group together similar instances
- Example: 2D point patterns



Objectives of “Clustering”

Finding groups of objects such that the objects within the same group are similar or related to one another, while being different from or unrelated to the objects in other groups.

- **Intra-Cluster Distance are Minimized:** Group objects (observations) that are very similar or homogeneous together (in same cluster).
- **Inter-Cluster Distance are Maximized:** Observations belonging to different clusters are different or heterogeneous
- Facilitates interpretation.



Most Common Clustering Algorithms

- **K-Means**
- Hierarchical Clustering
- Density Based Scan Clustering (DBSCAN)
- Gaussian Clustering Model

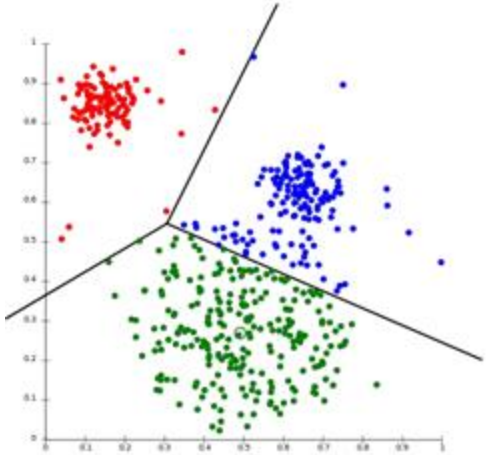
KMEANS CLUSTERING

K-Means

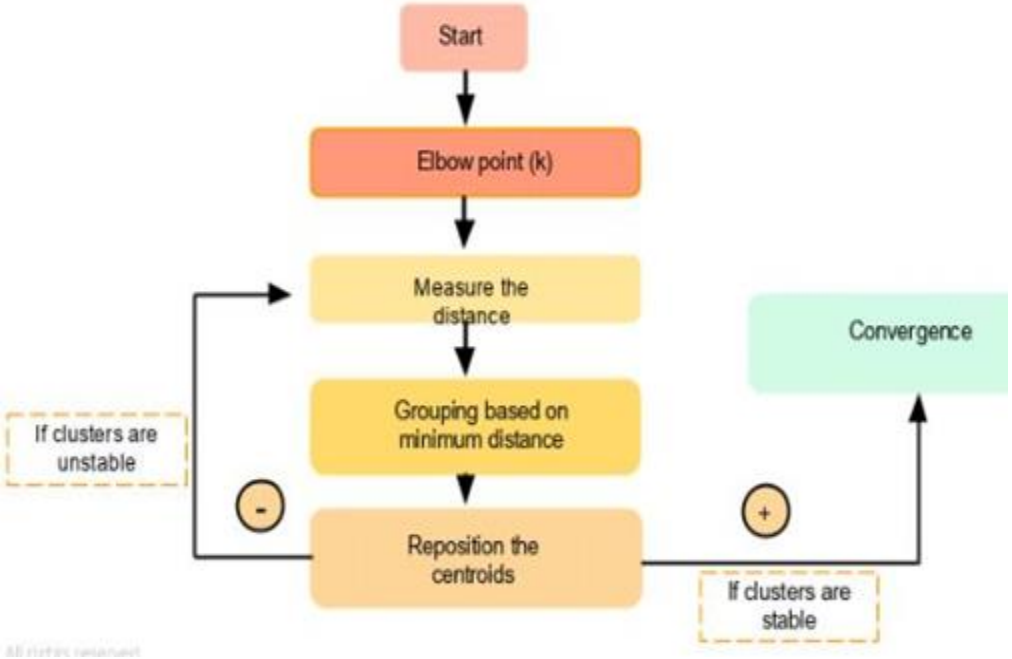
- As **unsupervised learning** algorithm
- An algorithm designed to assign clusters to each of your datapoints
- You begin with *k*, how many clusters you expect
- The algorithm randomly starts, then converges

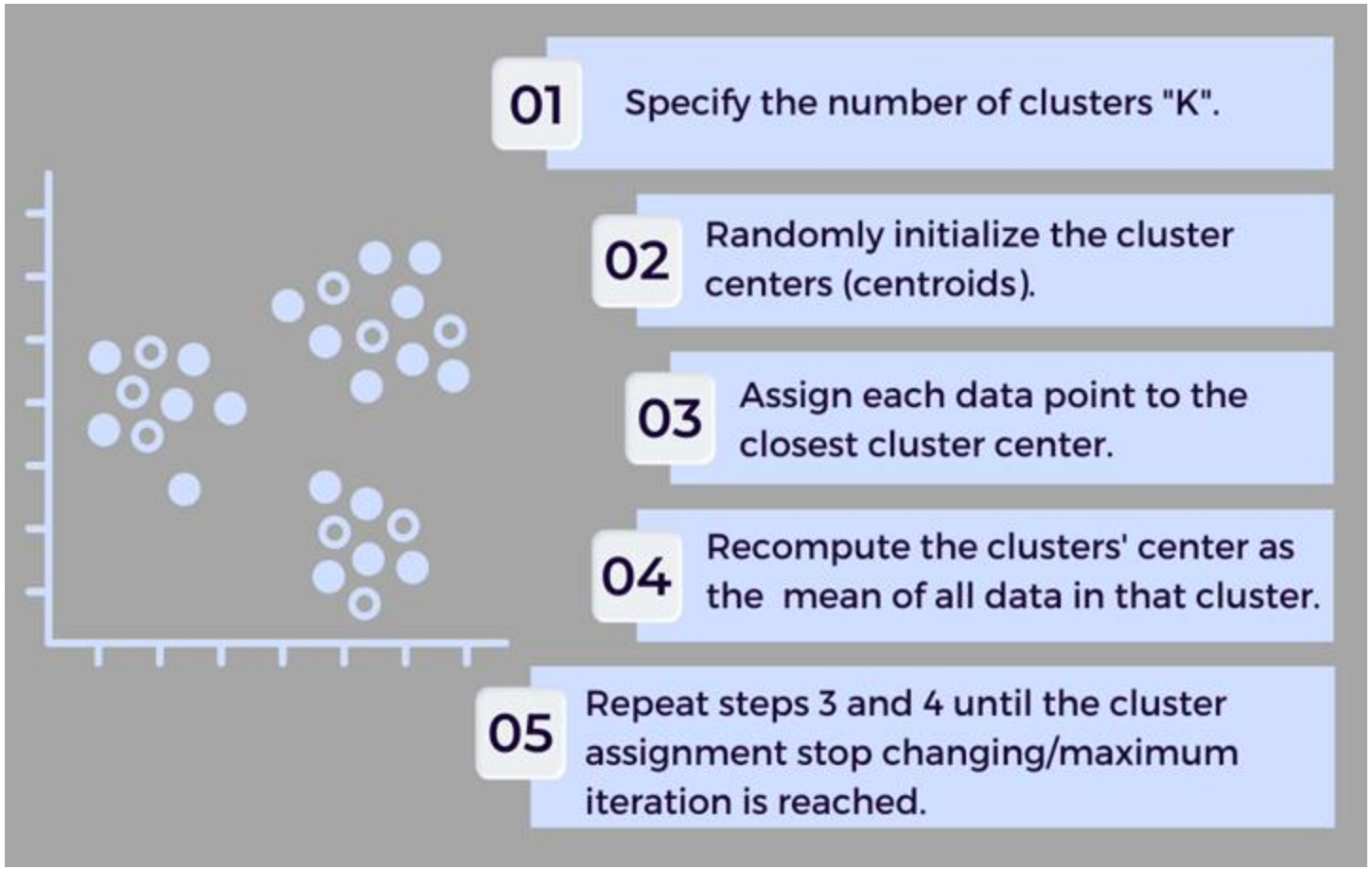
In Clustering, Quick Recap

- Input
  - a set S of n points in feature space
  - a distance measure specifying distance $d(x_i, x_j)$ between pairs (x_i, x_j)
- Output
  - A partition $\{S_1, S_2, \ldots, S_k\}$ of S



K-Means Clustering Algorithm: steps

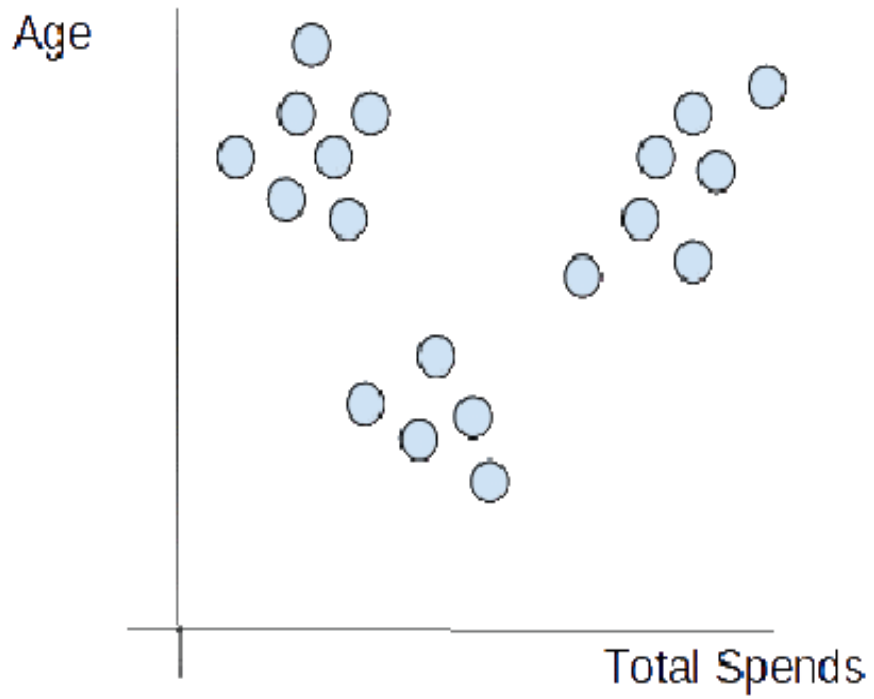




Algorithm 1 k-means algorithm

1: Specify the number k of clusters to assign.
2: Randomly initialize k centroids.
3: **repeat**
4: **expectation:** Assign each point to its closest centroid.
5: **maximization:** Compute the new centroid (mean) of each cluster.
6: **until** The centroid positions do not change.

How K-Means Works

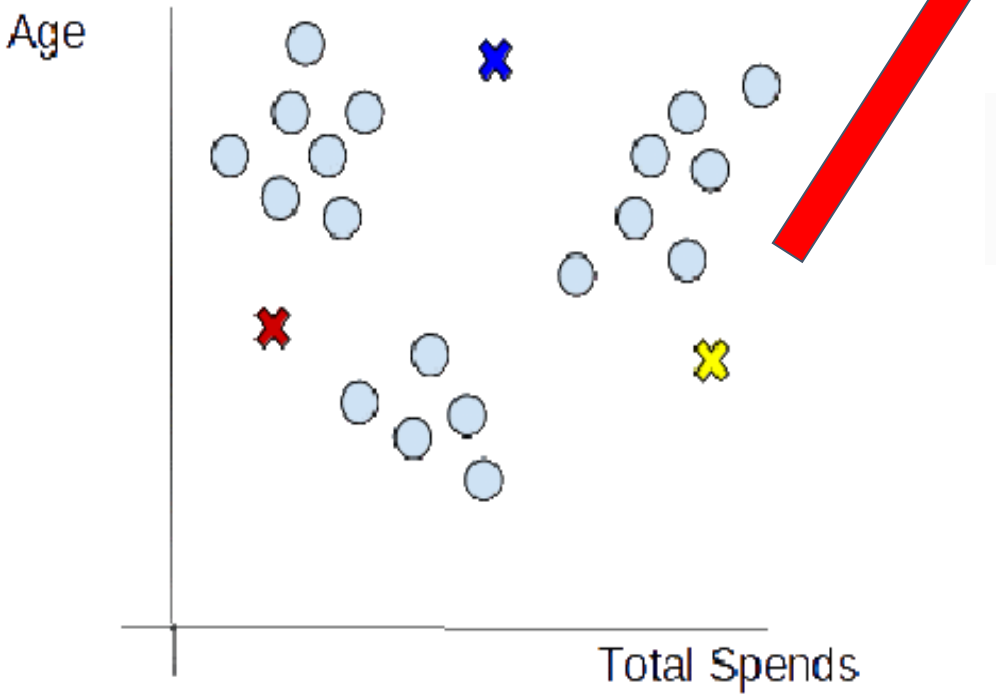


This is my Training Data

How K-Means Works

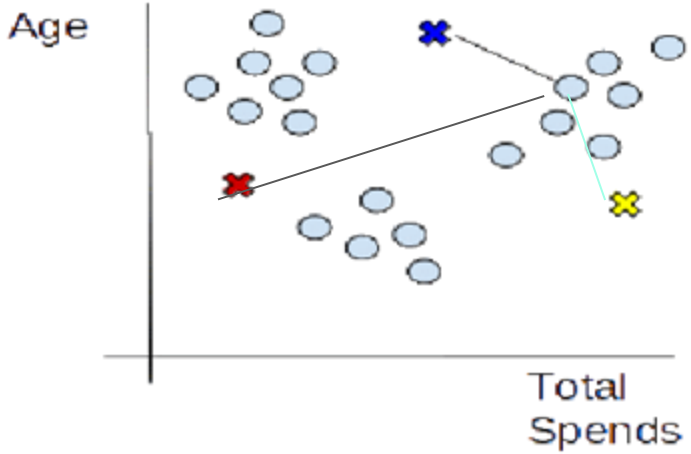
1. Let's Number of **K=3**

2. **Initialize** 3 Centroids randomly



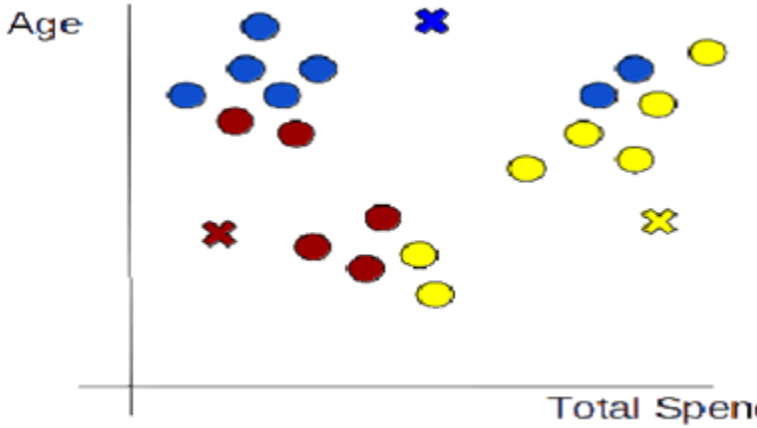
3. Assign Data Points to the Nearest Cluster (**2 STEPS**)

Step 3.1 Calculate the distance between each data point X and centroids



$$d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

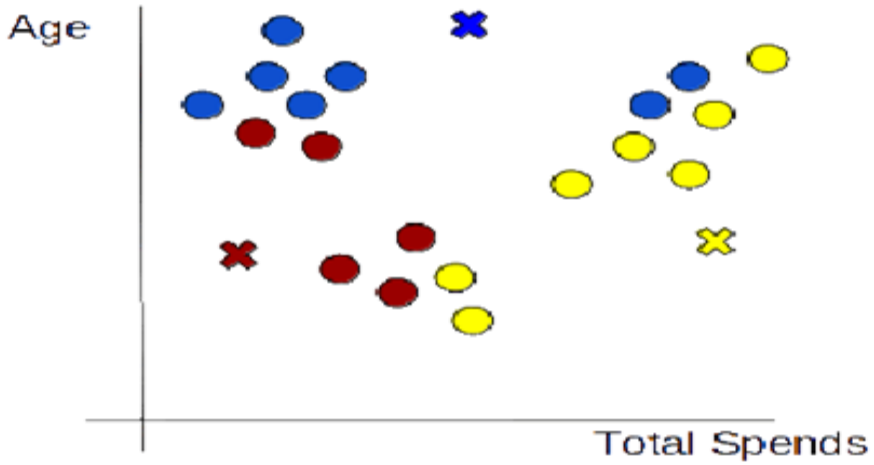
Step 3.2 Each point joins the **closest/nearest** cluster (based on its minimum distance to the centroid).



$$\underset{c_i \in C}{\arg\min}\ dist(c_i, x)^2$$

3 Assign Data Points to the Nearest Cluster (**2 STEPS**)

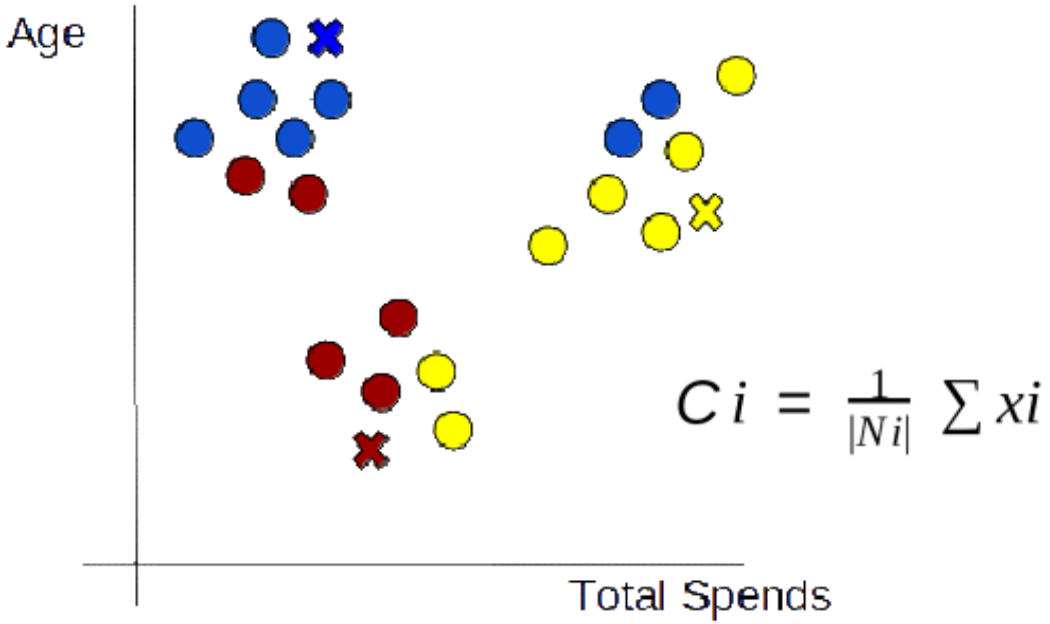
Calculate the distance between each data point X and centroids and Each point joins the **closest/nearest** cluster (based on its minimum distance to the centroid).



“PREVIOUS STEP FROM LAST SLIDE”

4

Re-initialize the centroids by calculating the average of all data points of that cluster.



‘N_i’ represents the number of data points X_i in ith cluster C_i.

In figure, for Red cluster,

C_{re}: (x’,y’)= ((x1+x2+...+x5)/5 , (y1+y2+...+y5)/5)

where N_i =5

How does K-means work?

5 **Repeat steps 3 and 4 until convergence**

Repeat **Steps 3 and 4** until optimal centroids and the assignments of data points to **correct clusters are not changing anymore.**

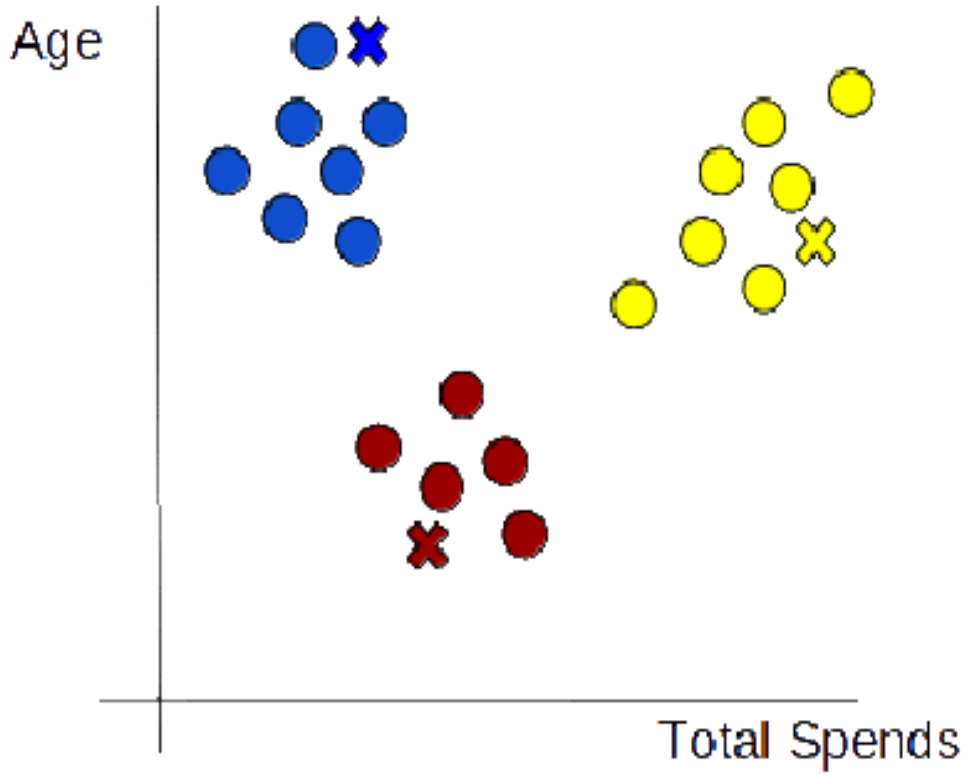


Figure: Repeat Step 3 and 4 until Convergence

The K-Means Algorithm

Training Data

K: number of clusters to discover

Algorithm 4 K-MEANS($\mathbf{D}$, K)

1: **for** $k = 1$ **to** K **do**
2: $\quad \boldsymbol{\mu}_k \leftarrow$ some random location // randomly initialize mean for kth cluster
3: **end for**
4: **repeat**
5: $\quad$ **for** $n = 1$ **to** N **do**
6: $\quad\quad z_n \leftarrow \text{argmin}_k \, ||\boldsymbol{\mu}_k - \boldsymbol{x}_n||^2$ // assign example n to closest center
7: $\quad$ **end for**
8: $\quad$ **for** $k = 1$ **to** K **do**
9: $\quad\quad \mathbf{X}_k \leftarrow \{ \boldsymbol{x}_n : z_n = k \}$ // points assigned to cluster k
10: $\quad\quad \boldsymbol{\mu}_k \leftarrow \text{MEAN}(\mathbf{X}_k)$ // re-estimate mean of cluster k
11: $\quad$ **end for**
12: **until** $\boldsymbol{\mu}$s stop changing
13: **return** z // return cluster assignments

Convergence/ Stopping Criteria for K-Means Clustering

- Centroids of newly formed clusters do not change → not learning any new pattern.
- Points remain in the same cluster.
- Maximum number of iterations are reached → how many times you want to run.

How to choose K?

- **If K is very large →** a cluster for every data point.
  - May defeats the purpose of clustering (assign cluster for every data point)
- **If K = 1**
  - One big cluster for the entire data set.

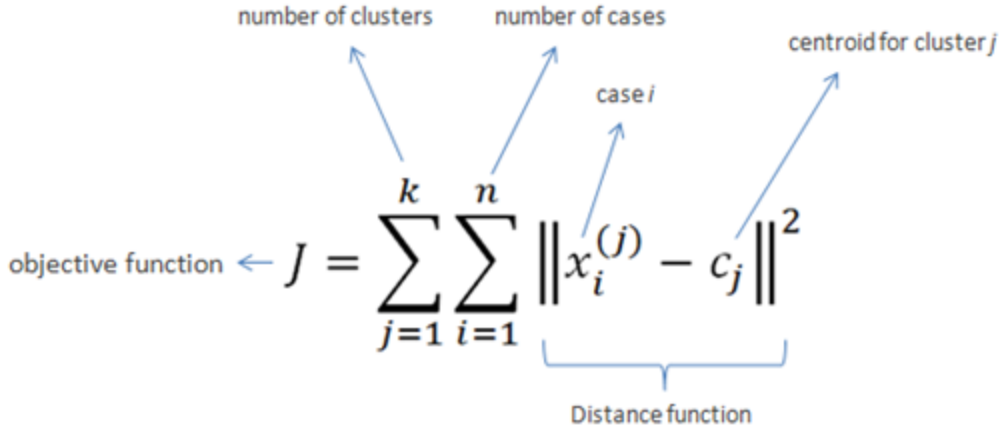
To select an ideal **K**, ensure that the identified clusters are distinct from each other.

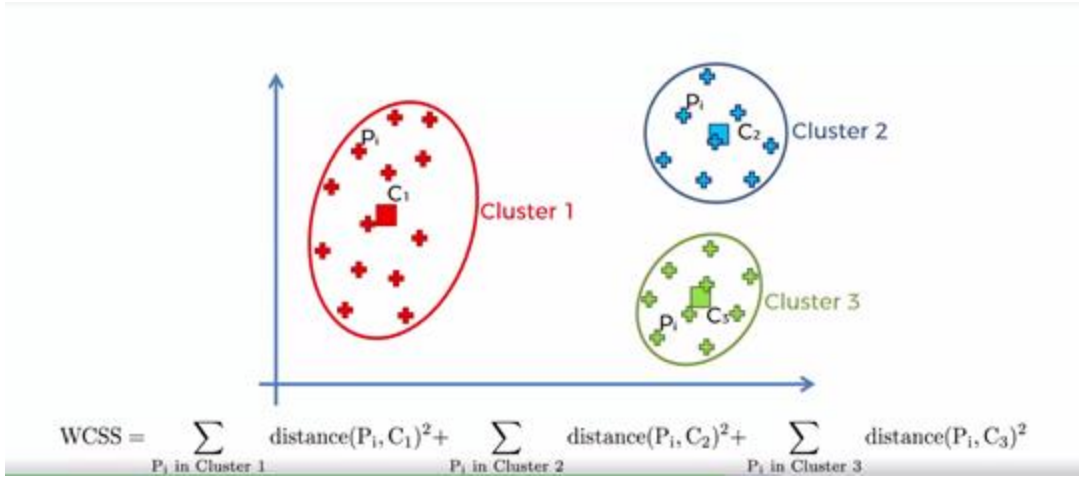
- The **distance** *from each point to its cluster's center* is **much smaller** than the **distance** *between the centers of different clusters*.

The Objective of K-Means Clustering

*"Minimize total **intra-cluster** variance, or **Within-Cluster Sum of Square (WCSS)or Inertia***

Where WCSS= sum of squares of the distances of each data point in all clusters to their respective centroids (WCSS).





Assumptions:

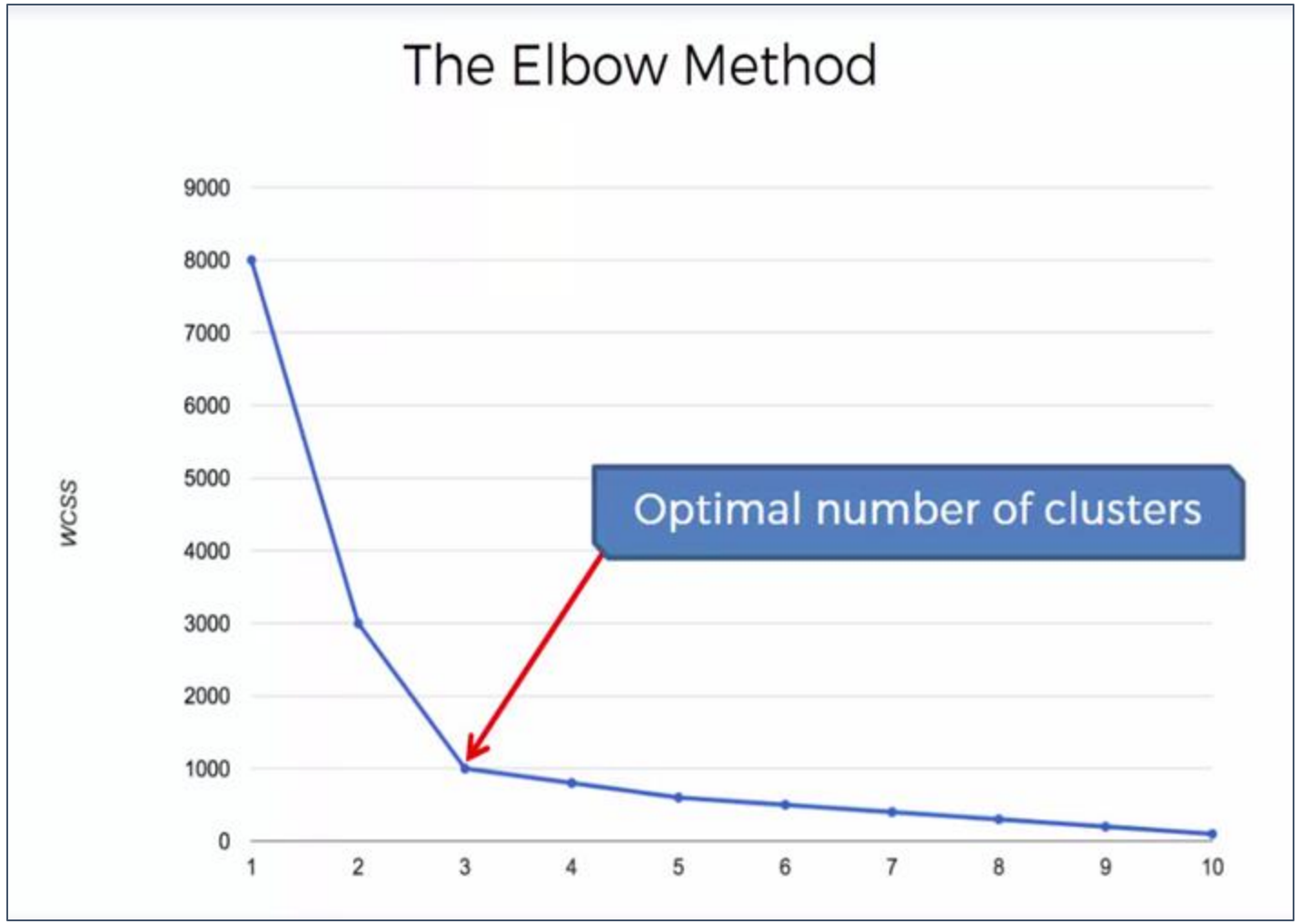
- Each observation belongs to at least one of the K clusters
- The clusters do not overlap
- Variation within each cluster is minimized.

$$WCSS = \sum_{C_k}^{C_n} \left(\sum_{d_i in C_i}^{d_m} distance(d_i, C_k)^2 \right)$$

Where,
C is the cluster centroids and d is the data point in each Cluster.

Finding the optimal number of clusters K - **Elbow Method**

- X axis: vary the number of clusters K
- Y axis -> for each value of K, calculate WCSS (Within-Cluster Sum of Square).



Observe:

- As the value of K increases, the sum of square of distances decreases for every step.
- But this decreases is very fast for the initial values of K, and then it slows down. This defines the right value of K.

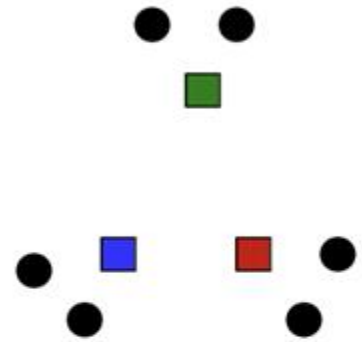
Another Method: The **silhouette score** method to determine the appropriate number of clusters

K-Means Properties

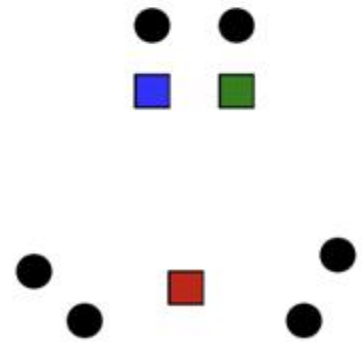
- Time complexity: O(KNL) where
  - K is the number of clusters
  - N is number of examples
  - L is the number of iterations
- K is a hyperparameter
  - Needs to be set in advance (or learned on dev set)
- Different initializations yield different results!
  - Doesn't necessarily converge to best partition
- "Global" view of data: revisits all examples at every iteration

Initialization

- K-means **algorithm** is a heuristic
  - Requires initial means
  - It does matter what you pick!



  - What can go wrong?
  - Various schemes for preventing this kind of thing: variance-based split / merge, initialization heuristics

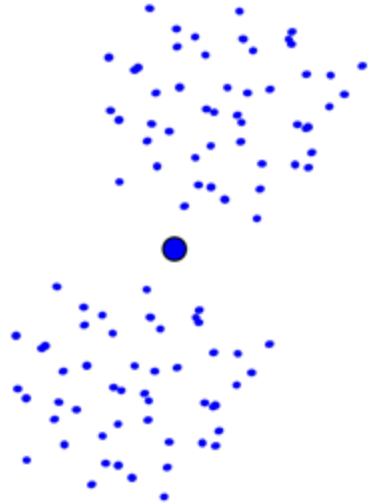


K-Means Getting Stuck

A local optimum:

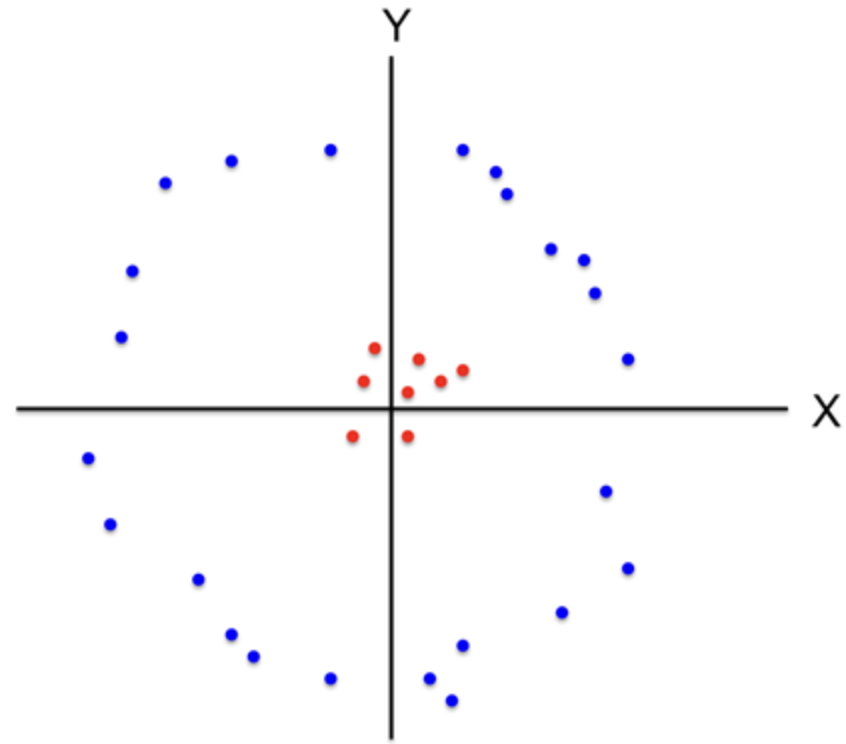


Would be better to have
one cluster here



… and two clusters here

K-means not able to properly cluster



SOME SOLUTIONS COULD BE

Run K-Means multiple times with different initializations

- Multiple runs (e.g., 10-50 times) of K-Means with different random centroid seeds.
- Choose the clustering result with the lowest total within-cluster variance (inertia).

Use K-Means++ initialization

- An improved method to initialize centroids more strategically rather than randomly.
- Spreads out the initial centroids to better cover the space and reduce chances of poor local minima.

Try alternative clustering algorithms

- consider Hierarchical Clustering or DBSCAN that do not rely on random centroid initialization

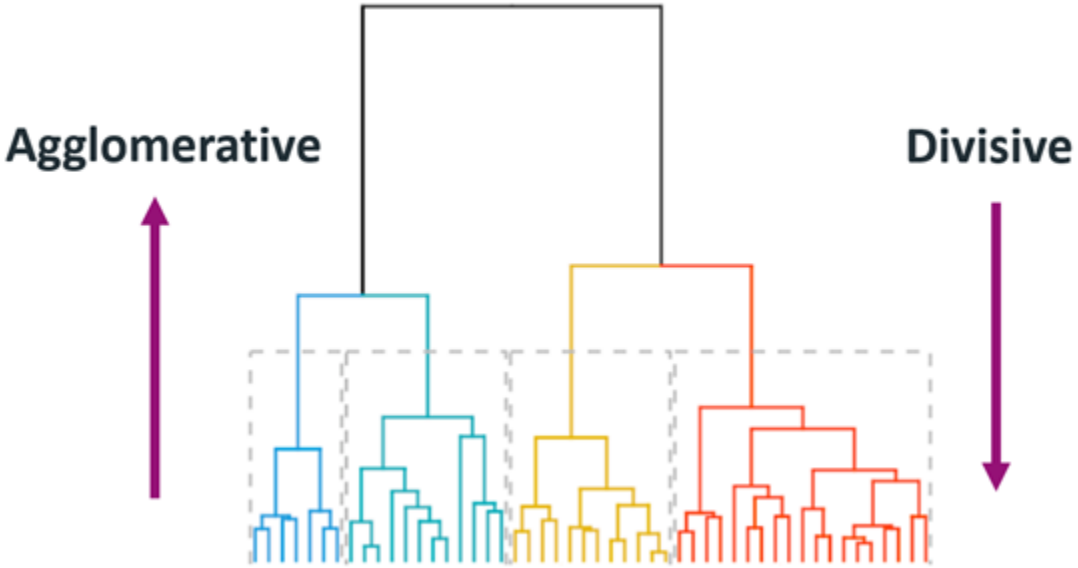
Hierarchical Clustering

Builds a hierarchy of clusters represented as a tree (dendrogram).

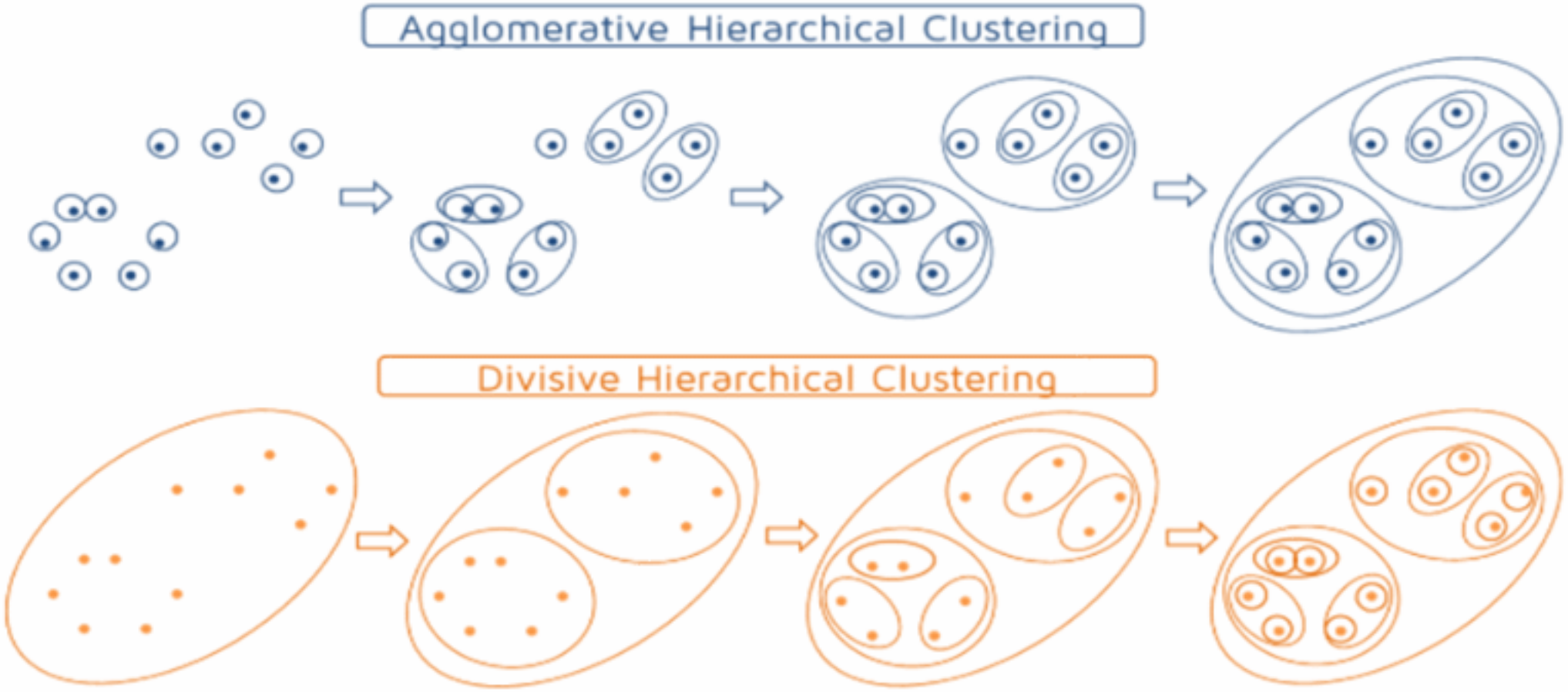
- Does not require pre-specifying the number of clusters (k).
- Two types:
  - **Agglomerative (bottom-up):** Start with each point as its own cluster and merge closest clusters step-by-step.
  - **Divisive (top-down):** Start with one cluster and recursively split.

Key Concepts:

- **Linkage methods:** Determine how to measure distance between clusters
  - Single linkage (min distance)
  - Complete linkage (max distance)
  - Average linkage (average distance)



Hierarchical Clustering



DBSCAN (Density-Based Spatial Clustering)

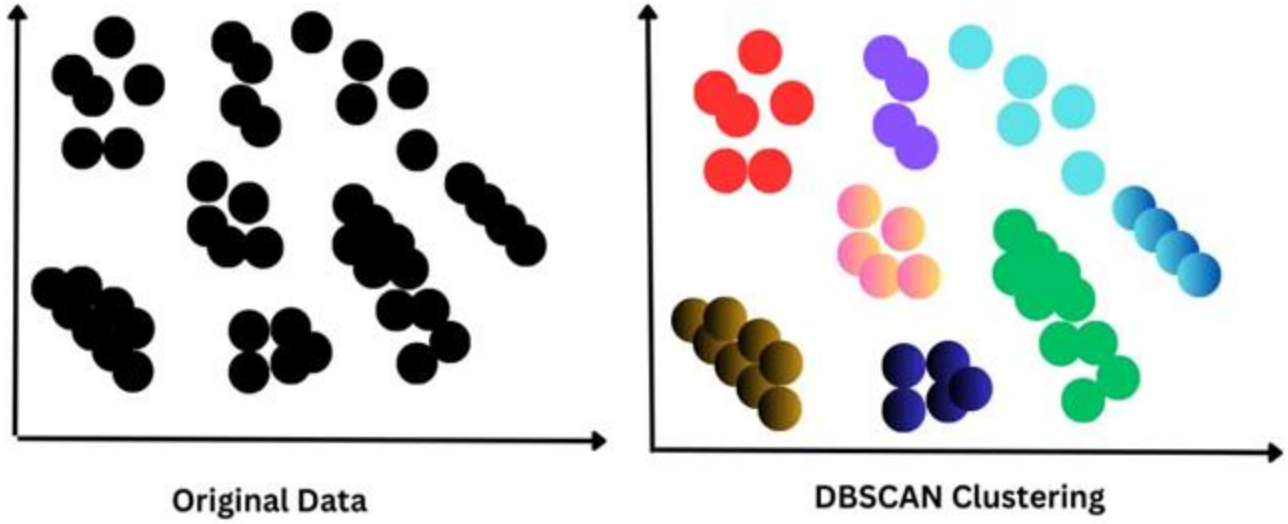
Density-based clustering algorithm that groups points closely packed together.

Can find clusters of **arbitrary shape** and identify **noise/outliers**.

Key Parameters:

- **ε (epsilon):** Radius to search for neighboring points.
- **minPts:** Minimum number of points required in an ε-neighborhood to form a dense region.

Density-Based Spatial Clustering of Applications with Noise (DBSCAN) groups data points based on their density, forming clusters while handling outliers effectively



DBSCAN: Core, Border & Noise Points

Core point: Forms & expands clusters; has ≥ minPts neighbors within ε.

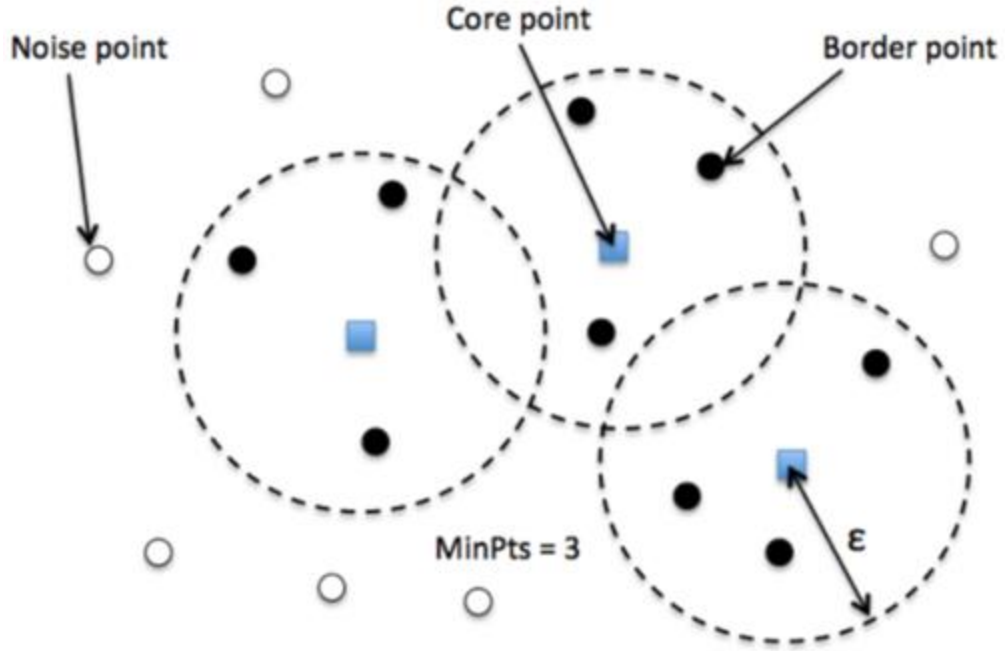
Border point: Defines cluster edges (Within ε of a core point but fewer than minPts neighbors.)

Noise: Neither core nor border point. Example: Isolated point, not in any group

Why Both Core & Border?

- Core: Ensures dense clusters grow
- Border: Smooths edges, avoids abrupt cuts

  → Together, they find natural-shaped clusters!



Hierarchical Clustering

Pros:

- Easy to visualize cluster relationships with dendrograms.
- Flexible, no need to specify *k* upfront.

Cons:

- Computationally expensive for large datasets.
- Sensitive to noise and outliers.

DBSCAN

Pros:

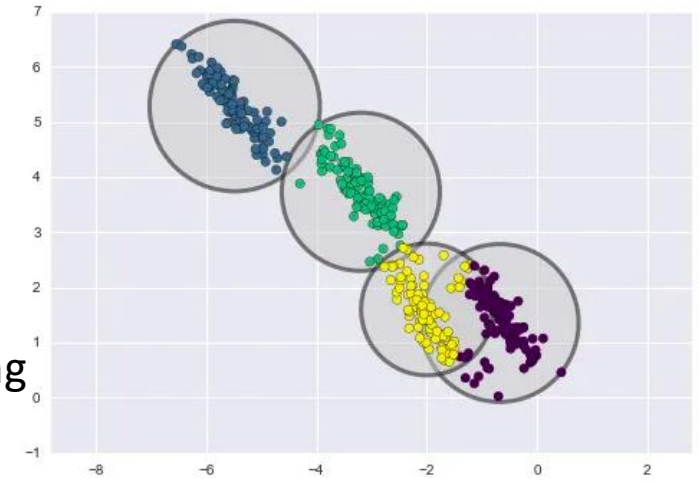
- Detects clusters with irregular shapes.
- Automatically detects noise/outliers.
- No need to specify number of clusters.

Cons:

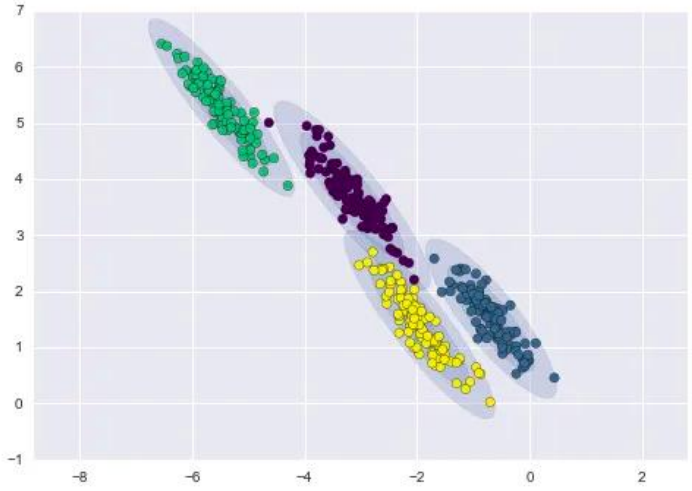
- Sensitive to parameter choice (ε and minPts).
- Performs poorly with varying densities.

Gaussian Mixture Models for Clustering

- Gaussian mixture models can be used to cluster unlabelled data in much the same way as k-means.
- There are, however, a couple of advantages to using Gaussian mixture models over k-means, like:
  - k-means does not account for variance. By variance means the width of the bell shape curve.
  - In two dimensions, variance determines the shape of the distribution.
  - When the data takes on different shape.



K-Means Clustering



GMM Clustering

Hard vs Soft Clustering Algorithms

- **K-Means**
- Hierarchical Clustering
- Density Based Scan Clustering (DBSCAN)
- Gaussian Clustering Model: Gaussian Mixture Models (GMM)

Hard Clustering
Each data point belongs to **exactly one cluster** (or is marked as noise in DBSCAN).

Soft Clustering
Each data point can belong to **multiple clusters with probabilities**

Even though **DBSCAN** and **Hierarchical** are more flexible in shape and structure than K-Means, they still assign points **deterministically**, so they fall under **hard clustering**.

Conclusion

- Clustering is a powerful unsupervised learning technique for discovering natural groupings in data.
- While K-Means is simple and fast, algorithms like Hierarchical Clustering and DBSCAN provide more flexibility to handle complex data shapes and noise.
- Choosing the right method depends on the data characteristics and the analysis goals.